



SCHOOL OF ADVANCED TECHNOLOGY
SAT301 FINAL YEAR PROJECT

Visualization of Music Expressiveness Analysis

Final Thesis

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Engineering

Student Name	:	Yishaoying Jia
Student ID	:	2253908
Supervisor	:	Shengchen Li
Assessor	:	Dechang Xu

Abstract

This paper presents Visualization of Music Expressiveness Analysis, a system for analyzing and visualizing musical expressiveness in Mozart piano sonata performances. The system addresses a core problem in current music analysis tools: the lack of an interpretable, interactive, score-integrated environment that allows music learners to explore hierarchical phrase structure in expressive piano performance. To this end, the system processes MIDI performance data synchronized with MusicXML scores to construct tempo curves (BPM), from which recursive minima energy point extraction identifies hierarchical phrase boundaries at multiple granularities. A dynamics-based peak-to-valley detection method provides complementary information by identifying phrase reset points in velocity curves. These analyses are presented through an interactive web-based visualization interface that enables users to browse musical structure at multiple granularity levels, observe tempo curve dynamics during playback, and switch between light and dark themes. Evaluation on the ASAP Mozart Piano Sonatas dataset demonstrates the system’s applicability to music education and research, with user studies confirming its effectiveness in conveying expressive performance patterns to non-expert users.

Keywords: music expressiveness; musical phrase detection; hierarchical structure analysis; Mozart piano sonata; piano performance visualization; performance analysis system

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Shengchen Li, for his invaluable guidance, patient mentorship, and continuous encouragement throughout the development of this project. His insightful feedback and constructive criticism have shaped both the technical direction and the intellectual rigour of this work.

My thanks also go to the creators of the ASAP dataset, whose frame-accurate score–performance alignments formed the empirical foundation of this study, and to the developers of the open-source libraries used in this project—music21, mido, NumPy, SciPy, Flask, and OpenSheetMusicDisplay—whose contributions to the music information retrieval community made this work possible.

My appreciation goes to the ten participants—six piano students and four piano teachers—who generously contributed their time and expertise to the user evaluation study. Their candid feedback was essential in assessing the practical value of this system.

Finally, I am deeply grateful to my family and friends for their unwavering support and understanding throughout my undergraduate studies at the School of Advanced Technology.

Contents

Abstract	i
Acknowledgements	ii
List of Tables	vii
List of Figures	viii
List of Acronyms	ix
1 Introduction	1
1.1 Background and Definition	1
1.2 Significance of Expressiveness Research	1
1.3 The Value of Visualization	2
1.4 Data Foundation: Scores and Performances	2
1.5 Problem Statement	3
1.6 Research Objectives and Main Contribution	3
2 Related Work and Theoretical Background	4
2.1 Music Expressiveness and Performance Analysis	4
2.2 Music Representation and Data Formats	5
2.3 Score-Performance Alignment	6
2.4 Music Energy and Phrase Boundary Analysis	6

2.5	Music Visualization Tools	7
3	System Design and Architecture	8
3.1	Overview	8
3.2	System Architecture	9
3.2.1	Data Layer	9
3.2.2	Analysis Layer	10
3.2.3	Presentation Layer	10
3.3	Design Rationale	11
3.4	Functional Requirements	11
3.5	Non-Functional Requirements	11
3.6	Data Flow	11
4	Algorithmic Methodology for Phrase Boundary Detection	13
4.1	Tempo Curve Construction	13
4.1.1	Note-Level Tempo	14
4.1.2	Tempo Curve	14
4.2	Recursive Minima Energy Point Extraction	14
4.2.1	Algorithm Overview	14
4.2.2	Local Minima Detection	15
4.2.3	Segment Energy Computation	16
4.2.4	Recursive Process	16

4.3	Dynamics-Based Peak-to-Valley Detection	16
4.3.1	Velocity Curve	17
4.3.2	Peak Detection	17
4.3.3	Valley Identification	18
4.3.4	Complementary Analysis	18
4.4	Complexity Analysis	18
4.5	Comparison with Existing Methods	19
5	System Implementation	19
5.1	Development Environment	19
5.2	Backend Implementation	20
5.2.1	Tempo Computation Module	20
5.2.2	Velocity and Peak-to-Valley Module	20
5.2.3	Score Parsing Module	21
5.2.4	Alignment Processing	21
5.3	Frontend Implementation	21
5.3.1	Score Visualization	22
5.3.2	Curve Visualization (Tempo and Velocity)	22
5.3.3	User Interface Components	23
5.4	API Design	24
5.5	Implementation Challenges	24

6	User Evaluation	25
6.1	User Evaluation	25
6.1.1	Evaluation Dataset	25
6.1.2	Participants	25
6.1.3	Evaluation Method	25
6.1.4	Usability Results	26
6.1.5	Practicality Results	26
6.1.6	Discussion	26
6.2	Discussion	27
6.2.1	Strengths	27
6.2.2	Limitations	27
7	Conclusion and Future Work	28
7.1	Summary of Contributions	28
7.2	Research Implications	29
7.3	Future Directions	29
7.4	Concluding Remarks	30
	References	30

List of Tables

2	Functional requirements specification	12
3	Non-functional requirements specification	12
4	Comparison between tempo-based minima detection and velocity-based peak-to-valley detection	18
5	Property comparison of the proposed method with existing phrase boundary detection methods	19
6	Development environment and technology stack	19
7	RESTful API endpoints exposed by the backend	24

List of Figures

1	System architecture diagram showing the three-layer architecture	9
2	Data flow diagram from file upload to visualization	13
3	Flowchart of the minima energy point extraction algorithm	15
4	Hierarchical structure tree showing multi-level boundaries	17
5	Minima Viewer main interface: (a) Initial state, (b) File loaded, (c) Analysis complete, (d) Dark theme	23
6	Hierarchical level switching demonstration: Level 1, Level 2, Level 3	23

List of Acronyms

Term	Initial components of the term
API	Application Programming Interface
ASAP	Aligning Scores with Performances
BPM	Beats Per Minute
CDN	Content Delivery Network
CSV	Comma-Separated Values
DTW	Dynamic Time Warping
HMM	Hidden Markov Model
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
MFCC	Mel-Frequency Cepstral Coefficients
MIDI	Musical Instrument Digital Interface
MIR	Music Information Retrieval
MusicXML	Music Extensible Markup Language
OSMD	OpenSheetMusicDisplay
REST	Representational State Transfer
RMS	Root Mean Square
SVG	Scalable Vector Graphics
UI	User Interface
XML	Extensible Markup Language

1. Introduction

1.1 Background and Definition

Music expressiveness—the subtle variations in timing, dynamics, and articulation that distinguish one performance from another—has long been a central topic in musicology and performance studies [1–3]. Unlike a mechanical rendition where every note is played identically, expressive performance involves systematic deviations that convey musical meaning and emotional content [4]. Understanding how performers achieve musical expression is valuable for multiple stakeholders: music scholars seek to document and theorize about performance practice, while piano educators and students benefit from explicit understanding of interpretive choices [5,6].

1.2 Significance of Expressiveness Research

In music education, expressiveness analysis provides concrete benefits for piano pedagogy. Understanding how performers shape phrases helps teachers guide students toward more musical playing [1,2]. Computational tools can make interpretive patterns explicit and measurable, enabling more targeted instruction [6,7]. Studies on music perception indicate that students who develop awareness of expressive devices—such as dynamic shaping, rubato, and articulation—achieve higher levels of musical communication [6,8].

In musicology, computational analysis of expressiveness enables researchers to document and compare performance traditions across different schools, eras, and individual artists. The growing availability of digitally aligned score-performance datasets provides new opportunities for large-scale analysis of how different pianists interpret the same works [9]. These applications motivate the development of tools that make expressiveness analysis accessible to both researchers and learners.

1.3 The Value of Visualization

Visualization connects computational music analysis to musical understanding [10]. Visual representations transform abstract musical measurements—such as tempo curves and energy profiles—into formats that align with how musicians naturally perceive phrase structure [11]. Interactive visualization enables exploration of hierarchical musical relationships across multiple time scales, supporting both detailed technical analysis and interpretive understanding [12]. This makes expressiveness analysis accessible not only to researchers with computational expertise but also to performing musicians, educators, and learners who would otherwise have no direct way to see the structures that shape musical expression.

1.4 Data Foundation: Scores and Performances

The widespread availability of digital sheet music and MIDI recordings has created new opportunities for automated performance analysis. MusicXML [13] provides a standardized format for exchanging sheet music between software applications, while MIDI (Musical Instrument Digital Interface) [14] encodes performance data including velocity (key strike intensity) and precise timing information. Computational approaches can process performance data at scale and reveal patterns that human listening tends to miss. The ASAP (Aligning Scores with Performances) dataset [9] exemplifies this trend, providing frame-accurate alignments between MusicXML scores and audio recordings for classical piano repertoire, including complete Mozart sonatas.

Mozart piano sonatas are well suited as a testbed for expressiveness analysis. Their well-documented structural organization—typical Classical period conventions of exposition, development, and recapitulation—provides clear expectations against which performers’ interpretive choices can be measured. The ASAP dataset further includes multiple performances of each work, enabling comparative analysis of how different pianists realize the same score.

1.5 Problem Statement

The core problem this thesis addresses is: *existing music analysis tools make it difficult for music learners to directly observe how performers interpret phrase structure across multiple time scales*. While MIDI velocity and timing data are readily available, translating these raw measurements into musical interpretations—most notably the identification of phrase boundaries, those “breathing points” that shape interpretive expression [15]—remains a substantial obstacle for non-expert users.

This problem manifests through three interrelated gaps in current approaches. First, a **semantic gap** exists between audio-based analysis and score structure: audio-based segmentation methods [16, 17] operate on acoustic features and lack integration with the symbolic notation that defines Western classical repertoire. Second, a **granularity gap** limits most existing methods to single-level boundary detection, unable to expose the hierarchical organization—from motif to phrase to section—that underlies a performer’s interpretive choices. Third, a **usability gap** separates research-oriented Music Information Retrieval (MIR) systems [18] from the learners and educators who would benefit most from them; general-purpose visualization tools [12] typically treat all notes uniformly, without surfacing the hierarchical structure that performers actually shape.

Together, these three gaps describe the absence of an interpretable, interactive, score-integrated tool for exploring hierarchical phrase structure in expressive piano performance.

1.6 Research Objectives and Main Contribution

To address this problem, the **main contribution of this thesis** is:

An interactive web-based visualization system that exposes hierarchical phrase structure in expressive piano performance, by integrating tempo- and dynamics-based analyses on score-aligned MIDI data and supporting multi-granularity exploration through a unified interface.

This contribution is realised through four components:

1. A **three-layer end-to-end architecture** spanning data parsing (MusicXML, MIDI, and alignment files), analytical computation (tempo curve construction and recursive minima energy point extraction), and web-based presentation, providing a complete pipeline from raw performance data to interactive visualization.
2. A **complementary tempo- and dynamics-based analysis** that combines the recursive minima energy point extraction method [19] applied to tempo curves with a peak-to-valley detection scheme on velocity curves, characterising phrase boundaries from two perspectives—timing variation and dynamic shaping.
3. A **hierarchical multi-granularity interface** that allows users to switch between levels of the recursively generated structure via a level slider, with synchronised score highlighting, tempo/velocity curve plotting, and system-by-system playback, making the otherwise abstract hierarchy directly explorable.
4. An **empirical validation** conducted on the ASAP Mozart Piano Sonatas dataset [9] with ten participants (six piano students and four piano teachers), confirming the system’s value as a pedagogical aid and clarifying its boundaries of applicability.

2. Related Work and Theoretical Background

2.1 Music Expressiveness and Performance Analysis

Music expressiveness refers to the subtle variations in timing, dynamics, and articulation that distinguish how performers interpret written scores. Unlike literal performance, where notes are played exactly as written, expressive performance involves systematic deviations that convey musical meaning [4–6]. These deviations are not random but follow patterns shaped by musical structure, genre conventions, and individual artistic choices [2, 20, 21].

Understanding expressiveness has implications for both music pedagogy and performance analysis. Studies have shown that phrase boundaries and breathing points are essential for how performers shape music [1, 2]. Research in music psychology has shown that both timing and dynamics carry substantial expressive information, with timing variations playing a particu-

larly salient role in emotional communication [22, 23]. Interpretive choices, whether realised through timing or intensity, tend to cluster at musical phrase boundaries [5, 15, 24]—a regularity that motivates the joint use of tempo and velocity features in the present work.

The field of Music Information Retrieval (MIR) provides tools for automated music analysis [12, 18]. Early work focused on content identification and retrieval, but more recent research has addressed performance analysis [6, 7]. Computational approaches to performance analysis span a broad methodological spectrum. At one end, explicit rule systems encode expressive principles symbolically, as exemplified by the KTH framework, which formalises phrasing, micro-timing, articulation, and tonal tension as performance rules [25]. At the other end, data-driven models infer expressive patterns from annotated corpora [26, 27], and recent advances in automatic music transcription [28], together with the growing availability of large performance corpora [29], have further expanded the empirical basis for such studies. Rather than encoding explicit rules or learning a generative model of expression, the present work occupies a complementary signal-structural niche: it derives interpretable descriptors directly from aligned symbolic–performance data, prioritising transparency and pedagogical accessibility over modelling capacity. This positioning motivates the choice of energy-based hierarchical analysis described in the following sections. Indeed, many existing approaches treat performances as audio signals rather than as interpretations of specific scores, missing the connection between notation and performance that is central to Western classical music.

2.2 Music Representation and Data Formats

Effective performance analysis requires both score information and performance data. Two formats are particularly relevant for this study.

MusicXML provides a standardized representation of musical notation in XML format [13]. Unlike plain MIDI, MusicXML captures the semantic structure of scores: measures, parts, voices, and expressive markings. This structured representation is essential for understanding the score positions of notes and their structural roles.

MIDI encodes performance data as time-stamped messages [14]. The velocity value (0–127) of each note-on message reflects the intensity of key strikes, which correlates with acoustic

loudness on acoustic pianos. MIDI timestamps provide precise timing information, enabling detailed analysis of temporal patterns. However, MIDI alone lacks the semantic structure of scores.

These complementary formats address each other’s limitations: MusicXML provides structure, MIDI provides performance data. Bringing them together requires establishing correspondence between score positions and performance timestamps.

2.3 Score-Performance Alignment

Score-performance alignment establishes the correspondence between symbolic notation and actual performance timing [30–32]. This is necessary because performers rarely play notes exactly at their notated times; expressive timing means actual timings deviate systematically from the score.

Automated alignment tools such as MATCH [33] use Dynamic Time Warping algorithms to align different renditions of the same piece, enabling large-scale comparative performance studies. The ASAP dataset [9] provides frame-level alignments between MusicXML scores and audio recordings, with annotations for multiple performances of each work. The alignments enable analysis at the note level, comparing how different performers interpret the same score.

This study uses the ASAP alignments to compute performance features at specific score positions. By mapping MIDI velocity and timing to score measures and beats, we can analyze how performers realize notated structure in their interpretations.

2.4 Music Energy and Phrase Boundary Analysis

Energy-based analysis provides a psychoacoustically grounded approach to understanding musical structure [15, 34]. Musical energy, reflecting perceived loudness, varies systematically with phrase organization [6, 20]. Research on musical tension perception demonstrates that accumulated energy correlates with listeners’ subjective experience of musical buildup and release [15]. Velocity curves—representing the intensity of key strikes—reveal phrase-level

dynamics: local peaks often correspond to phrase climaxes, while valleys between peaks often correspond to phrase boundaries where performers reduce dynamic intensity.

Computational methods for phrase boundary detection have evolved significantly [35], approaching the problem from several complementary feature perspectives. Audio-based methods detect novelty in spectral features such as MFCCs [16], providing the conceptual basis for one of the comparison baselines used in Chapter 4. Symbolic-melodic methods identify boundaries from intervallic and contour patterns in monophonic lines [36], an approach orthogonal to the polyphonic timing analysis pursued here. Timing-based methods, which most directly inform the present work, exploit tempo modulations through dynamic programming [37] and build on foundational beat-tracking research [38]. More recently, learning-based methods have been applied both to low-level event detection (e.g., convolutional networks for onset detection [?]) and to high-level music structure analysis [26]. The hierarchical method adopted in this work [19] belongs to the timing-based family but extends single-level segmentation through recursive analysis, yielding nested phrase boundaries that enable the multi-granularity browsing interface described in Chapter 3. By repeatedly finding local minima in tempo curves and splitting at those points, boundaries emerge hierarchically, providing an interpretable basis for structure analysis.

In addition to tempo-based minima analysis, dynamics-based peak-to-valley analysis offers a complementary approach. By detecting local peaks in velocity curves and identifying the valleys between consecutive peaks, this method captures phrase reset points where performers often reduce dynamic intensity. The combination of tempo-based and dynamics-based analyses provides phrase boundary information from two complementary perspectives.

2.5 Music Visualization Tools

Interactive visualization helps make analytical results accessible to musicians and researchers [10, 11]. Surveys of music visualization techniques document how visual representations render abstract musical data in forms that can be read against the score [10, 12], enabling exploration of hierarchical musical relationships across multiple time scales. Several mature technologies underpin web-based music visualisation in practice, ranging from client-side score renderers

to audio-driven feedback systems; the following discussion positions the present work with respect to each.

Among score rendering libraries, Verovio [39] and OpenSheetMusicDisplay [40] both produce SVG output from MusicXML; this work adopts OSMD due to its DOM-level cursor API, which simplifies the coupling between curve playback and on-score highlighting required by the present interface. A related but distinct research line concerns score following—the real-time alignment of an ongoing performance with a notated score [41–43]. While score following addresses synchronisation during live performance, the present system operates offline on already-aligned recordings, treating alignment as input rather than as a tracking problem to be solved. Beyond static score rendering, audio-driven visualisation systems [44] have explored real-time feedback for music analysis, but typically without integrating symbolic structure. The visualisation contribution of this work lies precisely in this integration: combining symbolic score rendering with performance-derived analytical overlays at multiple granularities.

3. System Design and Architecture

3.1 Overview

Visualization of Music Expressiveness Analysis is a web-based music expressiveness analysis system that integrates score rendering, tempo analysis, and interactive visualization. The system processes MusicXML scores synchronized with MIDI performance recordings to construct tempo curves (BPM), from which hierarchical minima energy points are extracted to identify phrase boundaries from tempo variation.

- **Input:** MusicXML score file, MIDI performance file, and annotation-based alignment file (.match or JSON format)
- **Processing:** Tempo curve construction, recursive minima extraction, and valley detection
- **Output:** Interactive web visualization with hierarchical phrase boundary navigation, system-by-system visual playback, and tempo curve plotting

- **Target Users:** Music learners, amateur musicians, and enthusiasts interested in understanding expressive performance patterns

3.2 System Architecture

The system follows a three-layer architecture that separates data processing, analytical computation, and presentation components, as illustrated in Figure 1.

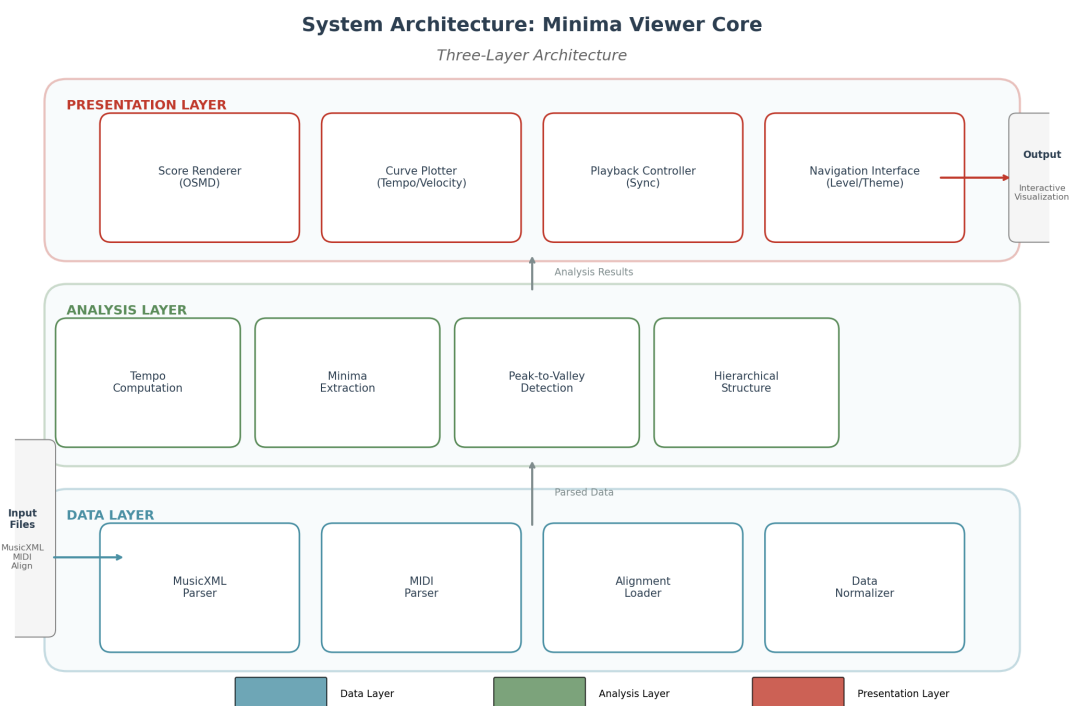


Figure 1: System architecture diagram showing the three-layer architecture

3.2.1 Data Layer

- Responsible for parsing and validating input files (MusicXML, MIDI, alignment)
- Provides a unified data interface for upper-layer modules
- Handles format conversion and data normalization
- Key modules:
 - **MusicXML Parser:** Extracts note attributes (pitch, duration, dynamics, position) using XML parsing

- **MIDI Parser:** Reads MIDI events (note-on, note-off, velocity) using the mido library
- **Alignment Loader:** Processes .match or JSON alignment files and provides index mapping between time and score position

3.2.2 Analysis Layer

- Performs tempo computation and minima extraction
- Implements the hierarchical phrase boundary detection algorithm
- Key modules:
 - **Tempo Computation Module:** Constructs tempo curves (BPM) from aligned MIDI timing and score position data
 - **Minima Extraction Module:** Recursively extracts hierarchical minima energy points for phrase boundary detection
 - **Peak-to-Valley Module:** Detects local peaks in velocity curves and identifies valleys between consecutive peaks
 - **Hierarchical Structure Generator:** Organizes extracted boundaries into multi-level tree structures

3.2.3 Presentation Layer

- Provides web-based interactive visualization
- Handles system-by-system visual playback synchronization
- Key components:
 - **Score Renderer:** Uses OpenSheetMusicDisplay (OSMD) to render MusicXML scores with minima highlighting
 - **Curve Plotter:** Canvas-based visualization of tempo and velocity curves with peak/valley annotations

- **Visual Playback Controller:** Drives system-by-system auto-cycling and synchronises the current system with curve highlighting updates
- **Navigation Interface:** Level slider, theme toggle, and language selector

3.3 Design Rationale

- **Three-Layer Architecture:** Separation of concerns enables independent development and testing of each layer; the analysis layer remains agnostic to presentation technology
- **Annotation-Based Alignment:** Chosen over automated alignment (e.g., DTW) to ensure frame-accurate correspondence between performance data and score positions [30,32,33]
- **Web-Based Visualization:** Cross-platform accessibility without plugin installation; enables deployment on local servers or cloud platforms [10, 11]
- **Recursive Minima Extraction:** Allows dynamic exploration of phrase boundaries at multiple granularities, supporting pedagogical and research use cases [19,34]

3.4 Functional Requirements

3.5 Non-Functional Requirements

3.6 Data Flow

The data processing pipeline consists of three stages: (1) Data Layer parses and validates input files, establishing correspondence between MIDI timestamps and score positions; (2) Analysis Layer computes tempo/velocity curves and extracts hierarchical minima; (3) Presentation Layer renders score with highlighted boundaries, plots curves, and handles user interactions.

Table 2: Functional requirements specification

ID	Requirement
F1	Score Loading and Rendering: Load MusicXML files and render readable notation with part/measure organization
F2	Tempo/Velocity Curve Computation: Calculate tempo and velocity curves from aligned MIDI data
F3	Minima Energy Point Extraction: Extract hierarchical minima energy points using recursive algorithm
F4	Hierarchical Browsing Interface: Navigate phrase boundaries at multiple granularity levels via level slider
F5	Peak-to-Valley Detection: Detect peaks in velocity curves and identify valleys between consecutive peaks for complementary phrase boundary analysis
F6	System-by-System Visual Playback: Auto-cycle through score systems with synchronised highlighting of the current system and corresponding curve segment
F7	Theme Switching (Light/Dark): Toggle between light and dark interface themes
F8	Multi-Format Alignment Support: Parse .match, JSON, and CSV alignment files with consistent index mapping
F9	Export Analysis Results: Export detected phrase boundaries and energy data for external analysis

Table 3: Non-functional requirements specification

Category	Requirement
Performance	Hierarchical level switching updates the score, curve, and highlighting within 200ms of slider change; energy computation completes within 2 seconds for movements with up to 5,000 notes
Usability	Intuitive user interface accessible to music learners without technical background; clear visual feedback for detected phrase boundaries
Portability	Cross-platform deployment via web browser (Chrome, Firefox, Safari, Edge); no native installation required
Scalability	Support for multi-movement sonatas with cumulative analysis across movements
Maintainability	Modular architecture enabling independent updates to parsing, analysis, or visualization components

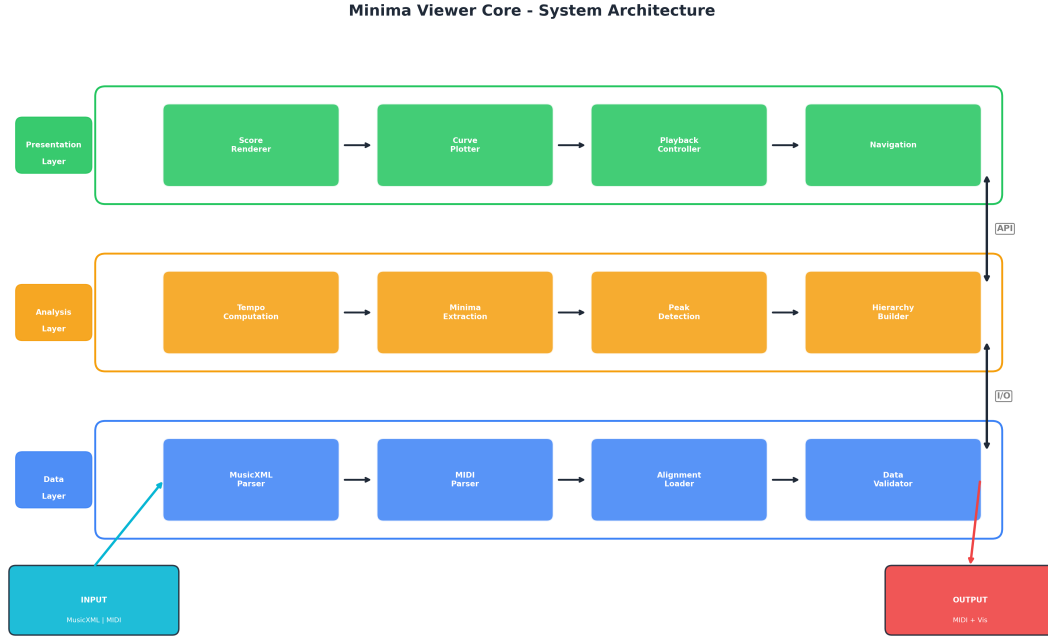


Figure 2: Data flow diagram from file upload to visualization

4. Algorithmic Methodology for Phrase Boundary Detection

This chapter presents the algorithmic methodology of this study. The system constructs tempo curves (BPM) from MIDI performance data synchronized with MusicXML scores and applies the recursive minima energy point extraction method [19] for hierarchical phrase boundary detection. A dynamics-based peak-to-valley detection method provides complementary analysis by identifying phrase reset points in velocity curves.

4.1 Tempo Curve Construction

The system constructs tempo curves to capture performance timing variations. Tempo represents the instantaneous speed of performance, computed from the ratio of score position changes to actual elapsed time.

4.1.1 Note-Level Tempo

Tempo at the note level is derived from MIDI timestamps and aligned score positions [5, 15]:

$$\text{tempo}_i = \frac{\Delta \text{position}}{\Delta \text{time}} = \frac{\text{pos}_i - \text{pos}_{i-1}}{\text{onset}_i - \text{onset}_{i-1}} \quad (\text{beats per second}) \quad (1)$$

Converted to BPM: $\text{tempo_BPM}_i = 60 \times \text{tempo}_i$

Research on rubato and tempo variations demonstrates that timing deviations are systematically related to musical structure [2, 5, 38]. Robustness filtering is applied to remove spurious values:

- Minimum time interval threshold: $\Delta t > 0.03$ seconds
- Valid tempo range: 20–400 BPM

4.1.2 Tempo Curve

The tempo curve $T = [t_1, t_2, \dots, t_n]$ represents the sequence of tempo values at aligned score positions, capturing the dynamic speed variations of a performance. Each tempo value corresponds to a specific score token (note or rest), enabling direct mapping between tempo variations and musical structure.

4.2 Recursive Minima Energy Point Extraction

4.2.1 Algorithm Overview

The system implements the recursive minima energy point extraction algorithm [19] with the following key steps, as illustrated in Figure 3:

1. **Input:** Tempo curve T (BPM sequence)
2. **Step 1:** Detect local minima in the current curve
3. **Step 2:** Split curve at minima positions into segments

4. **Step 3:** Merge adjacent segments pairwise
5. **Step 4:** Compute RMS energy of each merged segment
6. **Step 5:** Form next-level curve from segment energies
7. **Step 6:** Recurse on next-level curve
8. **Output:** Hierarchical minima at multiple granularity levels

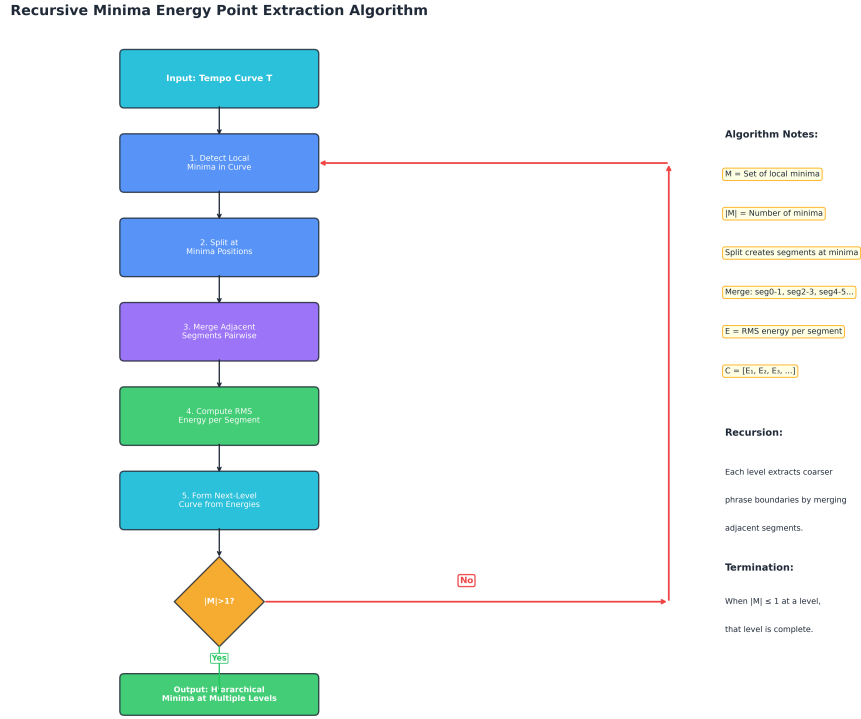


Figure 3: Flowchart of the minima energy point extraction algorithm

4.2.2 Local Minima Detection

A position k in curve C is a local minimum if:

$$C[k-1] > C[k] < C[k+1] \quad (2)$$

Subject to a prominence threshold θ :

$$\text{prominence}(k) = \min(C[\text{leftPeak}] - C[k], C[\text{rightPeak}] - C[k]) > \theta \quad (3)$$

Implementation uses `scipy.signal.find_peaks` with `prominence` parameter.

4.2.3 Segment Energy Computation

For each segment, RMS (Root Mean Square) energy is computed:

$$E_{\text{segment}} = \sqrt{\frac{1}{n} \sum_{i=1}^n C_i^2} \quad (4)$$

This provides a smoothed representation of tempo variations within each phrase segment.

4.2.4 Recursive Process

The algorithm produces boundaries at different granularities:

- **Level 0:** Original tempo curve $C_0 = T$, with minima M_0
- **Level 1:** After one merge, $C_1 = [\text{RMS of merged segments}]$, with minima M_1
- **Level 2:** After two merges, C_2 , with minima M_2
- **...**: And so on until $|M_k| \leq 1$

Each minimum $m \in M_i$ corresponds to a phrase boundary. An example of the resulting hierarchical boundary structure is illustrated in Figure 4.

4.3 Dynamics-Based Peak-to-Valley Detection

As a complementary approach to tempo-based minima detection, the system implements peak-to-valley detection on velocity curves to identify phrase boundaries based on dynamic variations.

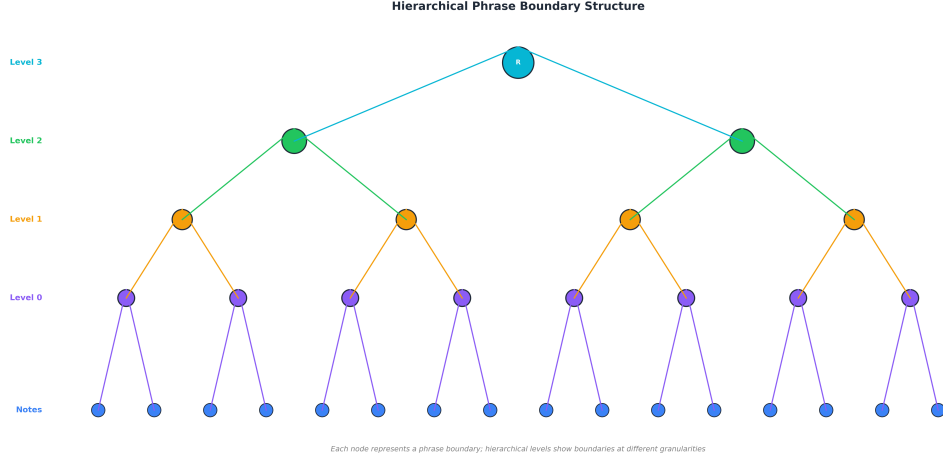


Figure 4: Hierarchical structure tree showing multi-level boundaries

4.3.1 Velocity Curve

The velocity curve $V = [v_1, v_2, \dots, v_n]$ represents MIDI velocity values (0–127) at aligned score positions:

$$v_i = \text{MIDI velocity of note } i \quad (5)$$

Velocity reflects the intensity of key strikes, correlating with perceived loudness.

4.3.2 Peak Detection

Local peaks in the velocity curve correspond to phrase climaxes—moments of maximum dynamic intensity:

$$v[k-1] < v[k] > v[k+1] \quad (6)$$

Subject to prominence threshold θ and minimum distance d between peaks.

4.3.3 Valley Identification

Valleys between consecutive peaks correspond to phrase reset points—moments where performers reduce dynamic intensity before building toward the next climax:

$$\text{valley}_i = \arg \min_{k \in [\text{peak}_i, \text{peak}_{i+1}]} V[k] \quad (7)$$

4.3.4 Complementary Analysis

Peak-to-valley detection provides a different perspective from tempo-based minima detection [6, 20]:

Table 4: Comparison between tempo-based minima detection and velocity-based peak-to-valley detection

Tempo-Based Minima	Velocity-Based Peak-to-Valley
Input: Tempo curve (BPM)	Input: Velocity curve (0–127)
Detects timing variations	Detects dynamic variations
Identifies slow-down points	Identifies intensity reset points
Hierarchical (multiple levels)	Single granularity

Research on piano performance dynamics shows that velocity variations correlate with phrase structure and emotional expression [8, 20, 45]. Both methods can be enabled simultaneously in the visualization to provide phrase boundary analysis from both timing and dynamics perspectives.

4.4 Complexity Analysis

- **Time Complexity:**

- Tempo curve construction: $O(n)$ where n is the number of aligned notes.
- Local minima detection: $O(m)$ where m is the curve length.
- Recursive extraction: $O(n \log n)$ in worst case.

- **Space Complexity:** $O(n)$ for storing the tempo curve and $O(k)$ for storing k extracted minima points.

4.5 Comparison with Existing Methods

Table 5: Property comparison of the proposed method with existing phrase boundary detection methods

Property	Proposed Method	MFCC-Novelty	HMM-Based
Input Type	MIDI + score alignment (tempo curve)	Audio	MIDI
Boundary Granularity	Hierarchical (multiple levels)	Single level	Single level
Output	Tempo curve + hierarchical boundaries	Boundaries only	Boundaries only
Score Integration	Yes (via MusicXML alignment)	No	No
Worst-case Complexity	$O(n \log n)$	—	—

5. System Implementation

5.1 Development Environment

Table 6: Development environment and technology stack

Component	Technology
Language	Python 3.9+ (backend), JavaScript ES6+ (frontend)
Backend Framework	Flask 2.x with RESTful API design, flask-cors
Score Parsing	music21 (MusicXML parsing)
MIDI Parsing	pretty_midi (MIDI file parsing)
Signal Processing	NumPy (array operations), SciPy (peak detection via <code>scipy.signal.find_peaks</code>)
Score Rendering	OpenSheetMusicDisplay (OSMD) 1.x
Data Visualization	HTML5 Canvas API
Deployment	Docker containerization, local server support

System Requirements: Modern web browser (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+), 4GB RAM minimum, internet connection for CDN resources.

5.2 Backend Implementation

The backend is implemented in Python, providing RESTful endpoints for file parsing, tempo computation, and minima extraction.

5.2.1 Tempo Computation Module

- **Tempo Calculation:** Uses NumPy arrays for efficient vectorized computation:

```
delta_beat = positions[1:] - positions[:-1]
delta_time = onset_times[1:] - onset_times[:-1]
tempo_curve = 60.0 * delta_beat / delta_time # BPM
```

- **Robustness Filtering:** Filters out spurious tempo values:
 - Minimum time interval: 0.03 seconds
 - Valid tempo range: 20–400 BPM
- **Recursive Minima Extraction:** Implements the Extract-Minima-Energy-Points algorithm [19] with:
 - `prominence`: Minimum prominence threshold (default: 0.01)
 - `distance`: Minimum sample distance between minima (default: 1)

5.2.2 Velocity and Peak-to-Valley Module

- **Velocity Curve:** Direct extraction of MIDI velocity values (0–127) from aligned notes.
- **Peak Detection:** Uses `scipy.signal.find_peaks` with prominence threshold.
- **Valley Identification:** For each consecutive peak pair, finds the local minimum (valley) between them.

5.2.3 Score Parsing Module

- **MusicXML Parsing:** Uses Python’s built-in `xml.etree.ElementTree`. Extracts note attributes including pitch, duration, dynamics, staff, and voice information.
- **MIDI Parsing:** Uses the `mido` library. Extracts note-on/ note-off events with velocity values and precise timestamps.
- **Code Snippet:** Key parsing function for MusicXML:

```
def parse_musicxml(xml_path):  
    tree = ET.parse(xml_path)  
    root = tree.getroot()  
    notes = []  
    for note_elem in root.iter('note'):  
        note = extract_note_attributes(note_elem)  
        notes.append(note)  
    return Score(notes=notes)
```

5.2.4 Alignment Processing

- **Multi-Format Support:** The system supports three alignment file formats [32]:
 - **.match Format:** Native format used by ASAP dataset
 - **JSON Format:** Flexible structure with timestamp–score position pairs
 - **CSV Format:** Tabular format with MIDI time and score position
- **Interpolation:** For timestamps between alignment entries, linear interpolation estimates corresponding score positions.

5.3 Frontend Implementation

The frontend is implemented as a single-page web application using modern JavaScript ES6+.

5.3.1 Score Visualization

- **OSMD Integration:** Uses OpenSheetMusicDisplay (OSMD) to render MusicXML scores in the browser with SVG output [40].
- **System-by-System Rendering:** The application parses the MusicXML structure and splits the score into individual systems (rows) based on the `new-system="yes"` XML attributes. Each system is rendered by a separate OSMD instance, enabling smooth scrolling through long pieces.
- **Minima Highlighting:** Color-coded beat regions highlight detected minima within measures on the score:
 - Beat 1: Light red overlay (`rgba(255, 100, 100, 0.15)`)
 - Beat 2: Light green overlay (`rgba(100, 255, 150, 0.15)`)
 - Beat 3: Light blue overlay (`rgba(100, 150, 255, 0.15)`)
 - Beat 4: Light yellow overlay (`rgba(255, 200, 100, 0.15)`)
- **Auto-Play Mode:** Upon analysis completion, the system automatically enters a system-by-system playback mode, cycling through each system at a configurable interval (default 5 seconds per system).

5.3.2 Curve Visualization (Tempo and Velocity)

- **Chart.js with Canvas Fallback:** The frontend uses Chart.js for rendering tempo curves with an automatic fallback to raw HTML5 Canvas drawing if Chart.js is unavailable.
- **Interactive Features:**
 - Theme-adaptive chart colors (light/dark mode)
 - Responsive scaling with the browser window
 - Position (beat) on x-axis, BPM on y-axis
- **Data Source:** Chart data is received from the backend's `/api/analyze` endpoint as

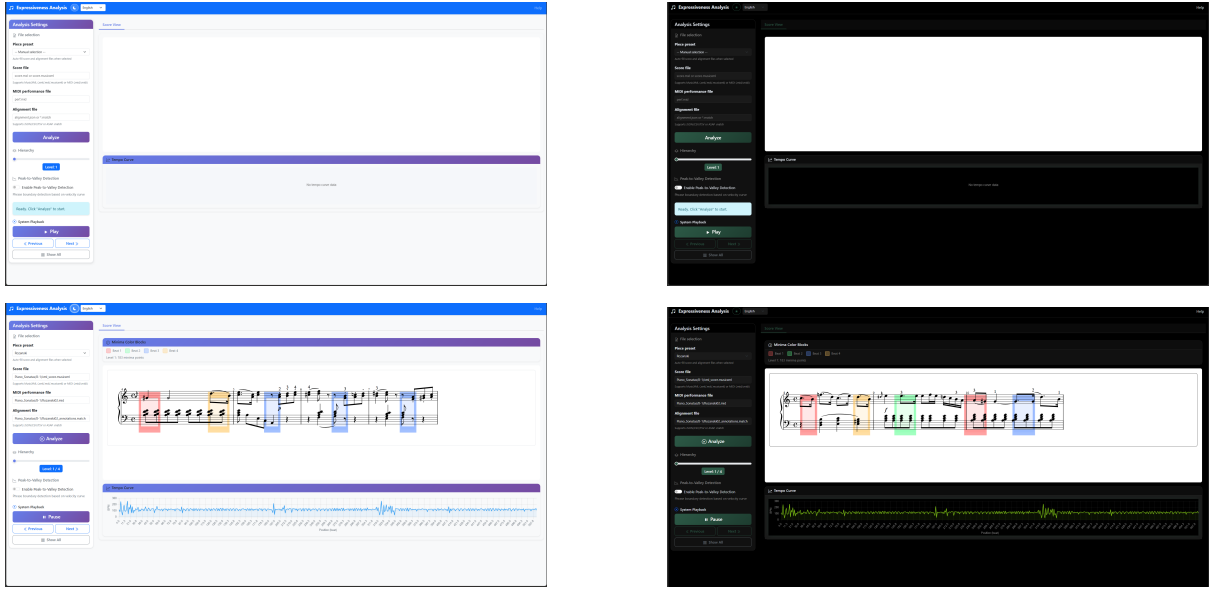


Figure 5: Minima Viewer main interface: (a) Initial state, (b) File loaded, (c) Analysis complete, (d) Dark theme

part of the comprehensive analysis response.

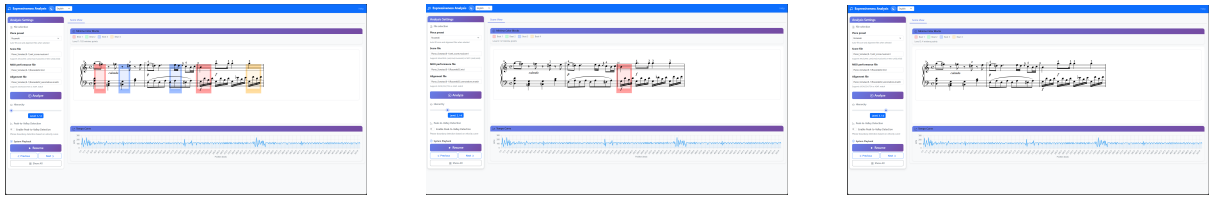


Figure 6: Hierarchical level switching demonstration: Level 1, Level 2, Level 3

5.3.3 User Interface Components

- **Piece Presets:** A dropdown menu provides quick access to pre-configured ASAP Mozart piano sonata datasets, eliminating the need for manual file path entry.
- **Level Slider:** Horizontal slider for selecting hierarchical browsing level (1 to maxLevel). Moving the slider updates all visualization components.
- **Peak-Valley Toggle:** A checkbox to enable/disable the peak-to-valley filtering of minima points.
- **Theme Toggle:** Switch between light and dark interface themes with persistent local-Storage preference.
- **Language Selector:** Dropdown for English/Chinese, with all UI strings externalized in

JSON localization files.

- **System Navigation:** Controls for navigating between systems (previous/next) and toggling auto-play mode.

5.4 API Design

The backend exposes a unified RESTful API endpoint for analysis, and a dedicated endpoint for serving MusicXML score files:

Table 7: RESTful API endpoints exposed by the backend

Endpoint	Description
POST /api/analyze	Receive score, MIDI, and alignment file paths; returns all analysis results including tempo curve, velocity curve, hierarchical minima, and peak-to-valley data
GET /api/score/{path}	Serve MusicXML score files from the server's score directory
GET /api/debug-routes	Debug endpoint listing all registered API routes

The `/api/analyze` endpoint accepts a JSON payload with file paths and returns comprehensive analysis data in JSON format. Upon successful analysis, the response includes:

- Parsed score tokens and performance tokens
- Alignment mappings between score positions and MIDI events
- Computed tempo curve and velocity curve data
- Hierarchical minima levels mapped to score positions
- Statistics (mean and standard deviation of velocity and tempo)

5.5 Implementation Challenges

- **Alignment Precision:** Minor discrepancies between MIDI timestamps and MusicXML timing required careful interpolation strategies.

- **Real-Time Rendering Performance:** Canvas-based curve rendering required optimization (requestAnimationFrame, double buffering) to maintain smooth scrolling.
- **Memory Management:** Large score files required streaming parsing and lazy loading to prevent memory overflow.

6. User Evaluation

6.1 User Evaluation

6.1.1 Evaluation Dataset

During the user evaluation sessions, participants interacted with the system using the **Piano_Sonatas** dataset [9], which includes:

- **4 Mozart Piano Sonatas:** K. 279, K. 310, K. 311, and K. 332
- **Multiple Performances:** 5–7 different pianists per sonata
- **Alignment Files:** Frame-accurate alignments in .match format
- **Musical Structure Annotations:** Harmonic analysis for structural reference

6.1.2 Participants

The evaluation recruited 10 participants:

- **6 Piano Students:** Grade 4 or above (National Piano Association of China)
- **4 Piano Teachers:** Professional piano instructors with teaching experience

6.1.3 Evaluation Method

Existing literature on the evaluation of computer systems for expressive music performance distinguishes three main categories of evaluation: comparison with real-performance mea-

surements, listening experiments, and production experiments [46]. Given the visualization-oriented nature of this study, where the goal is to support users' interpretive understanding rather than synthesise expressive performances, this evaluation adopts a listening-and-interview-based approach: participants used the system and provided feedback through online interviews covering **usability** and **practicality**.

6.1.4 Usability Results

All 10 participants unanimously agreed that the system has:

- **Clear UI Structure:** Well-organized and easy to navigate
- **Aesthetic Design:** Visually appealing presentation
- **Low Learning Curve:** Easily understood without extensive training

Regarding **usefulness for learning**:

- 4 of 6 piano students (67%) agreed the tool reduces time needed to study a specific pianist's performance style.
- All 4 piano teachers indicated it serves as a **supplementary teaching aid** to help students understand and imitate performance approaches.

6.1.5 Practicality Results

- 3 of 6 students (50%) would use it in future practice.
- All 4 teachers agreed it assists in teaching students who wish to learn a specific performer's style, but emphasized that phrase segmentation, tempo fluctuations, and dynamic variations remain **subjective decisions**, so the program cannot serve as the primary teaching method.

6.1.6 Discussion

Based on evaluation results:

1. **Strengths:** Clear and intuitive interface; participants identified the system as a supplementary tool for understanding performer styles, with teachers noting that it reduces the time needed to analyze a specific pianist’s interpretation.
2. **Limitations:** Relies on subjective musical judgment for phrase boundaries; not all users intend to use it for style imitation; the system is not intended to replace direct instruction.
3. **Suggestions raised by participants:** More customizable parameters; comparative analysis between multiple performers; interactive teaching features.

6.2 Discussion

6.2.1 Strengths

- **Direct Representation of Dynamics:** The tempo and velocity curves render timing and intensity variations as visible quantities, removing the need to infer them from audio listening alone [10]
- **Hierarchical Navigation:** Multi-level exploration supports both macro-level structure analysis and detailed phrase examination [34,47]
- **Synchronised Visual Playback:** System-by-system auto-cycling coordinates score highlighting with the corresponding curve segment, providing visual feedback that supports analytical reading without requiring audio synthesis
- **Accessibility:** Web-based implementation requires no installation and runs on standard browsers [12]

6.2.2 Limitations

- **Alignment Dependency:** Requires pre-computed score-performance alignment [9]
- **Single Instrument Focus:** Currently optimized for solo piano
- **Subjectivity of Boundaries:** Phrase boundaries in music are inherently subjective [1,2]
- **No Quantitative Comparison:** Provides visualization but not automated quantitative comparison between performances

7. Conclusion and Future Work

7.1 Summary of Contributions

This dissertation has addressed a single core problem: *the absence of an interpretable, interactive, score-integrated tool for exploring hierarchical phrase structure in expressive piano performance* [1, 6]. As argued in Chapter 1, this problem arises from three interrelated gaps in current music analysis tools—a semantic gap between audio-based analysis and score structure, a granularity gap that confines most methods to single-level segmentation, and a usability gap between research-oriented MIR systems and the learners they could serve.

To address this problem, the dissertation has presented a single integrated contribution—**Visualization of Music Expressiveness Analysis**, an interactive web-based visualization system that exposes hierarchical phrase structure in expressive piano performance through integrated tempo- and dynamics-based analyses on score-aligned MIDI data, with multi-granularity exploration supported through a unified interface.

This contribution is realised through four components:

1. An **end-to-end three-layer architecture** (data parsing → analytical computation → web visualisation) that establishes a complete pipeline from raw MusicXML/MIDI/alignment input to interactive output [5, 15].
2. A **complementary analytical scheme** that combines recursive minima energy point extraction on tempo curves [19] with peak-to-valley detection on velocity curves, capturing phrase boundaries from both timing and dynamic perspectives [20, 34, 45].
3. A **hierarchical multi-granularity interface** supporting interactive exploration of musical structure at multiple time scales, with synchronised score rendering, tempo/velocity curve plotting, and system-level playback control [7, 11].
4. An **empirical validation** on the ASAP Mozart Piano Sonatas dataset [9] with six piano students and four piano teachers, demonstrating use of the system as a supplementary pedagogical aid [48, 49] and clarifying its boundaries of applicability.

7.2 Research Implications

- The tempo curve analysis provides a timing-based approach to identifying phrase boundaries, complementing traditional musicological methods [37, 38].
- The hierarchical structure representation enables analysis at multiple time scales, supporting diverse use cases in music pedagogy [49] and musicology.
- The combination of tempo-based minima detection and dynamics-based peak-to-valley analysis suggests a multi-faceted approach to performance analysis [6, 35].
- The visualization-based evaluation approach supports qualitative assessment of performance expressiveness when quantitative ground truth is unavailable [10, 44].

7.3 Future Directions

1. **Real-Time MIDI Audio Playback and Note-Level Energy Visualisation:** The current implementation provides only system-by-system *visual* playback (auto-cycling between rendered systems). An extension is to add genuine MIDI audio synthesis with cross-browser Web Audio API support, building on established score-following techniques [41, 42] to keep audio output, on-score highlighting, and curve cursor synchronised. In parallel, the beat-level minima highlighting could be refined into a *note-level* energy visualisation, where individual notes on the score are colour-coded by their velocity-derived intensity, exposing per-note dynamic shaping at a finer granularity than the current measure-level overlay.
2. **Automated Score–Performance Alignment:** The current system depends on pre-computed alignment files (.match, JSON, or CSV) supplied by the ASAP dataset. Integrating a Dynamic Time Warping pipeline along the lines of MATCH [32, 33] would allow the system to accept arbitrary MIDI performances paired with a MusicXML score and produce the required alignment internally, removing the manual annotation requirement and broadening the range of usable performance recordings.
3. **Large-Scale Evaluation Across Repertoire and Performers:** The present user study covered four Mozart sonatas with ten participants. Running the same analysis pipeline

on broader Classical and Romantic repertoire (Beethoven, Chopin) [29], together with a larger cohort of piano students and teachers, would strengthen the empirical claims about the system’s pedagogical value and enable comparative studies of how different performers shape the same work.

7.4 Concluding Remarks

This dissertation demonstrates that hierarchical minima energy point analysis on tempo curves, supplemented by dynamics-based peak-to-valley detection on velocity curves [20,45], provides an interpretable approach to music phrase boundary detection and performance analysis [19]. The Visualization of Music Expressiveness Analysis system makes this analysis accessible through a web-based interface [10,11], contributing a tool for music learners and enthusiasts to examine expressive performance patterns in Mozart piano sonatas.

References

- [1] C. Palmer, “Music performance,” *Annual Review of Psychology*, vol. 48, no. 1, pp. 115–138, 1997.
- [2] E. F. Clarke, “Rhythm and timing in music,” in *The Psychology of Music*, D. Deutsch, Ed. Academic Press, 1999, pp. 473–500.
- [3] L. H. Shaffer, “Timing in musical performance,” *Annals of the New York Academy of Sciences*, vol. 423, no. 1, pp. 420–428, 1984.
- [4] J. A. Sloboda, *The Musical Mind: The Cognitive Psychology of Music*. Clarendon Press, 1985.
- [5] B. H. Repp, “Diversity and commonality in music performance: An analysis of timing microstructure in Schumann’s “Träumerei,”” *The Journal of the Acoustical Society of America*, vol. 92, no. 5, pp. 2546–2568, 1992.
- [6] G. Widmer and W. Goebel, “Computational models of expressive music performance: The state of the art,” *Journal of New Music Research*, vol. 33, no. 3, pp. 203–216, 2004.

- [7] C. E. Cancino-Chacón, M. Grachten, W. Goebel, and G. Widmer, “Computational models of expressive music performance: A comprehensive and critical review,” *Frontiers in Digital Humanities*, vol. 5, p. 25, 2018.
- [8] M. Zentner, D. Grandjean, and K. R. Scherer, “Emotions evoked by the sound of music: Characterization, classification, and measurement,” *Emotion*, vol. 8, no. 4, pp. 494–521, 2008.
- [9] F. Foscarin, A. McLeod, P. Rigaux, F. Jacquemard, and M. Sakai, “ASAP: A dataset of aligned scores and performances for piano transcription,” in *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR)*, Montréal, Canada, 2020, pp. 534–541.
- [10] H. B. Lima, C. G. R. dos Santos, and B. S. Meiguins, “A survey of music visualization techniques,” *ACM Computing Surveys*, vol. 54, no. 7, pp. 1–29, 2021.
- [11] R. Spence, *Information Visualization: Design for Interaction*, 2nd ed. Pearson / Addison Wesley, 2007.
- [12] M. Müller, *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.
- [13] W. Good and N. Scheirman, “MusicXML: An internet format for music notation,” Recordare LLC, [Online], 2001. [Online]. Available: <https://www.musicxml.org/>
- [14] MIDI Manufacturers Association, “The complete MIDI 1.0 detailed specification,” Los Angeles, CA, USA, 2001.
- [15] N. P. M. Todd, “The dynamics of dynamics: A model of musical expression,” *The Journal of the Acoustical Society of America*, vol. 91, no. 6, pp. 3540–3550, 1992.
- [16] J. T. Foote, “Automatic audio segmentation using a measure of audio novelty,” in *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, 2000, pp. 452–455.
- [17] J. P. Bello, L. Daudet, S. Abdallah, C. Duxbury, M. Davies, and M. B. Sandler, “A tutorial on onset detection in music signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 13, no. 5, pp. 1035–1047, 2005.

- [18] J. S. Downie, “Music information retrieval,” *Annual Review of Information Science and Technology*, vol. 37, no. 1, pp. 295–340, 2003.
- [19] S. Li, S. Dixon, and M. D. Plumbley, “A demonstration of hierarchical structure usage in expressive timing analysis by model selection tests,” in *Proceedings of the Chinese Control Conference*, 2018, pp. 3190–3195.
- [20] B. H. Repp, “A microcosm of musical expression: I. quantitative analysis of pianists’ timing in the initial measures of Chopin’s etude in E major,” *The Journal of the Acoustical Society of America*, vol. 104, no. 2, pp. 1085–1100, 1998.
- [21] R. Ashley, “Do [n’t] change a hair for me: The art of jazz rubato,” *Music Perception*, vol. 19, no. 3, pp. 311–332, 2002.
- [22] P. N. Juslin, “From mimesis to catharsis: Expression, perception, and induction of emotion in music,” in *Musical Communication*, D. Miell, R. MacDonald, and D. J. Hargreaves, Eds. Oxford University Press, 2005, pp. 85–109.
- [23] A. Gabrielsson and S. Lindström Wik, “Strong experiences related to music: A descriptive system,” *Musicae Scientiae*, vol. 7, no. 1, pp. 157–217, 2003.
- [24] R. Parncutt, “A perceptual model of pulse salience and metrical accent in musical rhythms,” *Music Perception*, vol. 11, no. 4, pp. 409–464, 1994.
- [25] A. Friberg, R. Bresin, and J. Sundberg, “Overview of the KTH rule system for musical performance,” *Advances in Cognitive Psychology*, vol. 2, no. 2–3, pp. 145–161, 2006.
- [26] O. Nieto and M. Farbood, “Identifying polyphonic musical patterns from audio recordings using music segmentation techniques,” in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, Taipei, Taiwan, 2014, pp. 411–416.
- [27] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [28] E. Benetos, S. Dixon, Z. Duan, and S. Ewert, “Automatic music transcription: An overview,” *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 20–30, 2019.
- [29] K. Kosta, O. F. Bandtlow, and E. Chew, “MazurkaBL: Score-aligned loudness, beat, and expressive markings data for 2000 Chopin mazurka recordings,” in *Proceedings of the*

- 4th International Conference on Technologies for Music Notation and Representation (TENOR)*, 2018, pp. 85–94.
- [30] P. Cano, A. Loscos, and J. Bonada, “Score-performance matching using HMMs,” in *Proceedings of the International Computer Music Conference (ICMC)*, 1999, pp. 441–444.
 - [31] N. Orio, S. Lemouton, and D. Schwarz, “Score following: State of the art and new developments,” in *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*, 2003, pp. 36–41.
 - [32] M. Müller, F. Kurth, and M. Röder, “An efficient algorithm for music synchronization,” in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2004, pp. 275–282.
 - [33] S. Dixon and G. Widmer, “MATCH: A music alignment tool chest,” in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2005, pp. 492–497.
 - [34] F. Lerdahl and R. Jackendoff, “An overview of hierarchical structure in music,” *Music Perception*, vol. 1, no. 2, pp. 229–252, 1983.
 - [35] O. Nieto, G. J. Mysore, C. Wang, J. B. L. Smith, J. Schlüter, T. Grill, and B. McFee, “Audio-based music structure analysis: Current trends, open challenges, and applications,” *Transactions of the International Society for Music Information Retrieval*, vol. 3, no. 1, pp. 246–263, 2020.
 - [36] E. Cambouropoulos, “The local boundary detection model (LBDM) and its application in the study of expressive timing,” in *Proceedings of the International Computer Music Conference (ICMC)*, 2001, pp. 112–115.
 - [37] C. Chuan and E. Chew, “A dynamic programming approach to the extraction of phrase boundaries from tempo variations in expressive performances,” in *Proceedings of the 8th International Society for Music Information Retrieval Conference (ISMIR)*, Vienna, Austria, 2007, pp. 305–308.
 - [38] E. D. Scheirer, “Tempo and beat analysis of acoustic musical signals,” *The Journal of the Acoustical Society of America*, vol. 103, no. 1, pp. 588–601, 1998.

- [39] RISM Digital Center, “Verovio: Music notation engraving library,” [Online], 2023. [Online]. Available: <https://www.verovio.org/>
- [40] OpenSheetMusicDisplay Contributors, “OpenSheetMusicDisplay: Render MusicXML in the browser,” [Online], 2023. [Online]. Available: <https://opensheetmusicdisplay.org/>
- [41] R. B. Dannenberg, “Real-time scheduling and computer accompaniment,” in *Current Directions in Computer Music Research*, M. Mathews and J. Pierce, Eds. MIT Press, 1989, pp. 225–261.
- [42] C. Raphael, “Music plus one and machine learning,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2010, pp. 21–28.
- [43] A. Cont, “ANTESCOFO: Anticipatory synchronization and control of interactive parameters in computer music,” in *Proceedings of the International Computer Music Conference (ICMC)*, 2008, pp. 33–40.
- [44] M. Graf, H. C. Opara, and M. Barthet, “An audio-driven system for real-time music visualisation,” in *Proceedings of the 150th Audio Engineering Society Convention*, Online, May 2021, p. Paper 10498.
- [45] B. H. Repp, “A microcosm of musical expression: II. quantitative analysis of pianists’ dynamics in the initial measures of Chopin’s etude in E major,” *The Journal of the Acoustical Society of America*, vol. 105, no. 3, pp. 1972–1988, 1999.
- [46] R. Bresin and A. Friberg, “Evaluation of computer systems for expressive music performance,” in *Guide to Computing for Expressive Music Performance*, A. Kirke and E. R. Miranda, Eds. Springer, 2012, pp. 141–162.
- [47] E. Terhardt, “Gestalt principles and music perception,” in *Auditory Processing of Complex Sounds*, W. A. Yost and C. S. Watson, Eds. Lawrence Erlbaum Associates, 1987, pp. 157–166.
- [48] S. G. Nielsen, “Learning strategies in instrumental music practice,” *British Journal of Music Education*, vol. 16, no. 3, pp. 275–291, 1999.
- [49] H. Meissner and R. Timmers, “Teaching young musicians expressive performance: An experimental study,” *Music Education Research*, vol. 21, no. 1, pp. 20–39, 2019.